

NETPULSE

Incident Response Platform

A Self-Built Enterprise-Grade Network Monitoring & Incident Management System

Shyam Kumar D

Aspiring Cloud Infrastructure Engineer

AWS CLOUD • NETWORKING • SECURITY

Contents

A complete map of the NetPulse project report

01	Executive Summary	03
02	Architecture Overview	06
03	Core Components	08
04	Data Flow	08
05	Step-by-Step Build Walkthrough	11
06	Challenges & Resolutions	27
07	Conclusion & Key Takeaways	30
08	Future Plans	32
09	Technology Stack Summary	30
10	Glossary & Acronyms	33

Executive Summary

What NetPulse is, how it is built, and the scale of the demo environment it operates today.

NetPulse Incident Response Platform is a self-built, full-stack Network Operations Center (NOC) application that detects outages across a simulated global network, automatically opens and correlates incidents, tracks Service Level Agreement (SLA) compliance, and gives operators a single console to diagnose and resolve problems. It was designed and built from scratch to reproduce the core workflow of commercial NOC/SOC tooling — detection, classification, assignment, and resolution — inside one cohesive TypeScript codebase, rather than relying on prebuilt monitoring SaaS products.

49

NODES MONITORED

5

GLOBAL SITES

86%

SLA COMPLIANCE

15ms

AVG. LATENCY

The system follows a modern three-tier architecture: a React 18 and TypeScript single-page frontend styled with Tailwind CSS, a Node.js and Express backend written in TypeScript, and a Redis-backed caching and publish/subscribe layer with an automatic in-memory fallback for local development when Redis is unreachable. Data is persisted through the Prisma ORM, which targets PostgreSQL in the Dockerized production topology and gracefully falls back to a local SQLite database for development without Docker. Real-time updates — new alerts, incident state changes, device health flips, and live metrics — are pushed to every connected browser over a Socket.IO WebSocket channel, so operators see outages the moment the monitoring engine detects them, without refreshing the page.

Nine functional modules make up the operator-facing product: the NOC Dashboard, Devices Inventory, the Alert Console, the Incident Room, the Incident Command Queue, the Topology Map, Analytics & KPIs, SLA & Maintenance, User Management, and the Activity Log — covering everything from fleet-wide health monitoring to role-based access control and a full audit trail.

At its core, NetPulse implements a simple but complete operational loop:



A background monitoring engine polls every registered device on a fixed interval using real network primitives (ICMP ping, TCP port probing, HTTP health checks, and DNS resolution, or live

agent telemetry where an agent is deployed). When a device fails a check, an alert-correlation and root-cause-analysis (RCA) engine decides whether the failure is a new, independent outage or a downstream symptom of a device that has already failed upstream in the topology, and either opens a new incident or attaches the alert to the existing one. From there, engineers pick incidents up from the Incident Command Queue, work them in the Incident Room, and the platform's SLA monitor continuously checks elapsed acknowledgement and resolution time against configurable policy thresholds, flagging breaches automatically.

In its current demo/test configuration, NetPulse is monitoring 49 nodes spread across five simulated international branches — the HQ Data Center, the London Office, the Tokyo Branch, the New York Tech Hub, and Sydney Operations — giving the dashboards, topology map, and analytics views a realistic, multi-site dataset to work with. Key differentiators over a loose collection of scripts and spreadsheets include automated multi-device alert correlation, configurable SLA policies per incident priority, an auditable per-incident operations timeline, and role-based access control across four operator roles.

HONEST FRAMING

This report documents NetPulse honestly as a personal, self-built project: it is not a production deployment used by a real organization, and its network data is simulated rather than sourced from live enterprise hardware. It is, however, a fully working, end-to-end platform with a real database schema, a real WebSocket layer, real network-checking code, and a real correlation/RCA engine — built to demonstrate the same architectural thinking that underpins commercial network-monitoring and incident-management products.

Architecture Overview

A three-tier system — frontend, backend, and a resilient data & cache layer — orchestrated with Docker Compose for production-style runs.

Architecture Overview

NetPulse is organized as two independently deployable services plus a data and cache tier, orchestrated together via Docker Compose for production-style runs, or run directly with Node.js for local development.

Frontend

REACT 18 + TYPESCRIPT

Backend

NODE.JS + EXPRESS

Data Layer

PRISMA + POSTGRESQL

◆ Frontend

The frontend (`frontend/src`) is a React 18 + TypeScript application bootstrapped with Vite. Styling is implemented with Tailwind CSS using a dark, glass-panel visual theme, visible throughout the NOC Dashboard, Alert Console, and Incident Room screens. Data fetching and caching is handled by TanStack React Query, charts are rendered with Recharts, and icons come from Lucide. Two React context providers anchor the app: **AuthContext.tsx**, which manages the signed-in user's session and role, and **SocketContext.tsx**, which owns a persistent Socket.IO client connection and exposes live alert, incident, device, and metric events to any page that subscribes to them. Page-level components in `frontend/src/pages` map directly onto NetPulse's nine functional modules, with shared chrome supplied by `components/Layout.tsx`.

◆ Backend

The backend (`backend/src`) is an Express application written in TypeScript, compiled and run in development with ts-node under nodemon, and listening on port 5000. Incoming traffic passes through helmet (security headers), cors, and an express-rate-limit policy capped at 300 requests per 15-minute window per client before reaching any route handler. REST endpoints are grouped by domain under `/api` — auth, devices, alerts, incidents, topology, analytics, agent, sla-policies, maintenance, users, and logs — each implemented as its own Express router in `backend/src/routes`. A lightweight `/health` endpoint reports service status for uptime checks.

◆ Data Layer

Persistence is handled through Prisma ORM against a schema (`backend/prisma/schema.prisma`) of seventeen models covering the full incident-management domain: User and RefreshToken for authentication, Site and Device for the monitored estate (with DeviceLink describing topology edges and DeviceMetric storing time-series telemetry), Alert and the Incident / IncidentDevice / IncidentTimeline / IncidentComment family for the incident workflow, MaintenanceWindow and SlaPolicy for operational policy, and Notification, ActivityLog, and AlertRule for auditing and

threshold-based alerting. In the Dockerized topology this schema targets PostgreSQL; for local, Docker-free development it targets a SQLite file with no code changes required.

◆ Caching & Publish/Subscribe

Redis provides caching and cross-process pub/sub in production. `backend/src/config/redisClient.ts` wraps the Redis client so that a failed or absent Redis connection never crashes the server: a connection error is caught, logged, and the cache module transparently swaps to an in-memory Map combined with a Node EventEmitter, so the rest of the application continues to function identically whether or not Redis is present. This fallback is visible directly in the backend console output during startup.

◆ Real-Time Communication

`backend/src/sockets/socketManager.ts` initializes a Socket.IO server on top of the same HTTP server as the REST API and exposes broadcast helpers — `broadcastAlert`, `broadcastIncident`, `broadcastDeviceUpdate`, and `broadcastMetric` — that the monitoring engine, alert-correlation engine, and SLA checker call whenever state changes. The frontend's `SocketContext.tsx` subscribes to these events and updates the relevant React Query caches, which is what makes the Alert Console, NOC Dashboard, and Incident Room update live, reflected in the UI's WS CONNECTED / WS OFFLINE status pill, without a manual page refresh.

DEPLOYMENT TOPOLOGY

Docker Compose orchestrates PostgreSQL, Redis, the Express backend, and an Nginx-hosted React frontend for production-style runs; the same codebase falls back cleanly to SQLite and in-memory caching for a Docker-free local development loop.

03 / PRODUCT SURFACE

Core Components

Nine operator-facing modules, each mapped to a dedicated frontend page and backend API domain.

NOC Dashboard

Fleet-wide totals, health distribution, and a rolling seven-day trend of active warnings versus confirmed outages, sourced from Dashboard.tsx.

Devices Inventory

The full device catalogue with hostname, IP, vendor/model, site, live telemetry health, uptime percentage and latency — Devices.tsx against /api/devices.

Alert Console

A live, filterable alarm feed with severity and status filters and inline Acknowledge / Resolve / Escalate actions — Alerts.tsx against /api/alerts.

Incident Room

The single-incident workspace showing RCA diagnosis, affected device list, discussion thread, and an auditable operations timeline.

Incident Command Queue

The cross-incident triage list with workflow-state and priority filters and live SLA acknowledgement/resolution status — Incidents.tsx.

Topology Map

An interactive WAN overview plus per-site LAN drill-down across all five branches — Topology.tsx against /api/topology.

Analytics & KPIs

MTTR, MTTA, overall SLA compliance, per-site availability, daily incident frequency, and top outage-driving devices and sites.

SLA & Maintenance

Configuration of Critical/High/Medium/Low priority SLA policies and scheduling of maintenance windows that suppress alerting on a target device.

User Management

Role-based access control across ADMIN, ENGINEER, NOC_OPERATOR, and VIEWER roles — Users.tsx against /api/users.

Activity Log

A chronological, searchable audit trail of authentication events, configuration changes, and incident actions — ActivityLogs.tsx.

04 / PIPELINE

Data Flow

The monitoring engine (`backend/src/services/monitoringEngine.ts`) drives the entire pipeline. Every ten seconds it loads every registered device and, for each one, either reads the latest telemetry reported by a NetPulse Agent (if one is configured for that device) or performs live

network checks — ICMP ping, TCP port checks, an HTTP health-check URL, and a DNS lookup — implemented in `backend/src/services/networkChecker.ts`. Devices inside an active maintenance window are skipped for alerting purposes. Each poll writes a new DeviceMetric row, updates the device's rolling uptime percentage, and broadcasts the device and metric update over WebSocket.

Any check failure is handed to the alert-correlation engine (`backend/src/services/alertCorrelation.ts`), which creates an Alert row, then walks the device's parent chain (`findHighestFailedAncestor`) to determine whether a device further up the topology is already down. If it is, the new alert is linked to that device's existing open incident instead of opening a duplicate; if not, a new Incident is created with a sequential identifier (INC-0001, INC-0002, ...) via `generateNextIncidentId`, and the failing device is recorded as the incident's root cause. This is the mechanism that keeps a single upstream failure — for example a distribution switch — from generating a separate incident for every server behind it.

In parallel, a second scheduler in `backend/src/index.ts` (`startSlaPolicyChecker`) runs every thirty seconds, loads every incident that is still open, compares its elapsed age against the `ackLimitSeconds` and `resLimitSeconds` defined on its assigned SlaPolicy, and — the first time either threshold is crossed — flags `slaBreachAck` or `slaBreachResolve`, writes an explanatory IncidentTimeline entry, and re-broadcasts the incident so every connected client immediately shows the breach.

```
poll(10s) → networkChecker.check(device) → on fail
  alertCorrelation.findHighestFailedAncestor(device)
    if ancestor down → attach Alert to existing Incident
    else → generateNextIncidentId() → open new Incident
slaPolicyChecker(30s) → compare elapsed vs ackLimitSeconds / resLimitSeconds → flag
breach
```

Build Walkthrough

A module-by-module tour of NetPulse, pairing each operator screen with the backend code and data model that drives it.

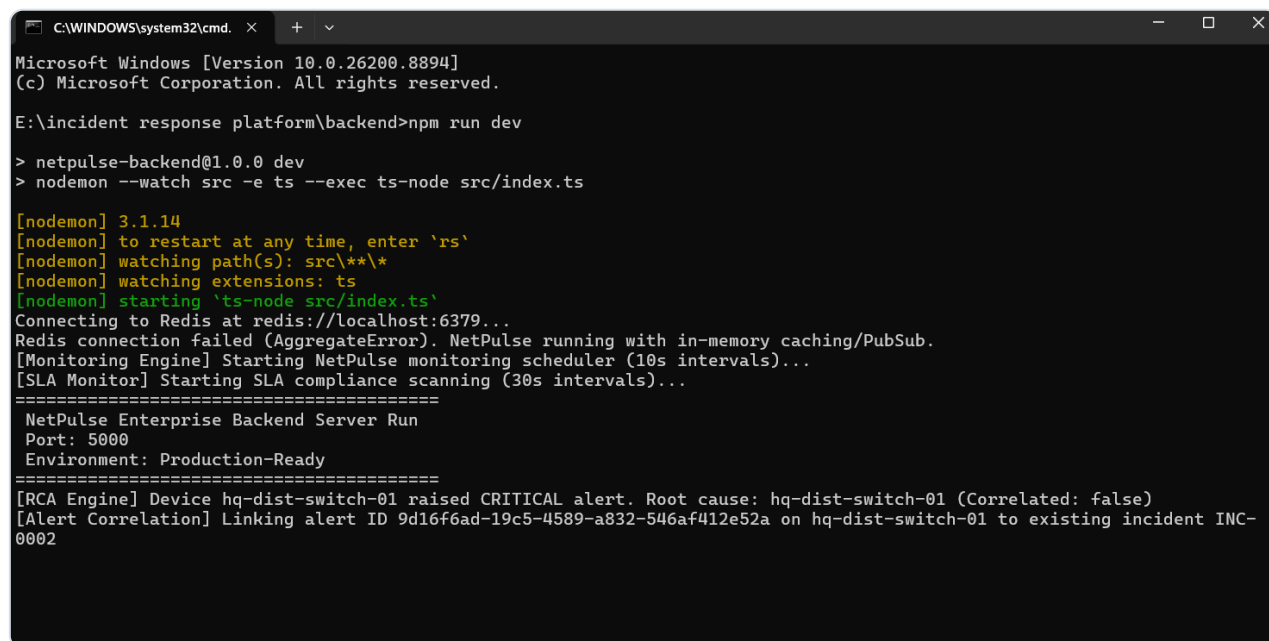
Step-by-Step Build Walkthrough

The following section walks through NetPulse module by module, in the order an operator would encounter them.

◆ 5.1 — Backend Startup, Redis Fallback & Monitoring Engine Boot

Starting the backend with `npm run dev` launches nodemon, which watches `backend/src` and runs the entry point (`src/index.ts`) through ts-node. On boot, the process attempts to connect to Redis at the URL configured in `REDIS_URL` ; when Redis is not reachable — as in this local run — the connection attempt fails gracefully and the cache layer switches to in-memory caching and pub/sub instead of crashing the process, exactly as designed in `redisClient.ts` .

Immediately afterward, the `bootstrap()` function in `index.ts` starts the two background schedulers: the Monitoring Engine on a ten-second interval and the SLA Monitor on a thirty-second interval, then binds the HTTP and Socket.IO server to port 5000. Subsequent log lines show the RCA engine and alert-correlation engine reacting to a device failure in real time — in this capture, `hq-dist-switch-01` raising a CRITICAL alert and being linked to existing incident INC-0002.



```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

E:\incident response platform\backend>npm run dev

> netpulse-backend@1.0.0 dev
> nodemon --watch src -e ts --exec ts-node src/index.ts

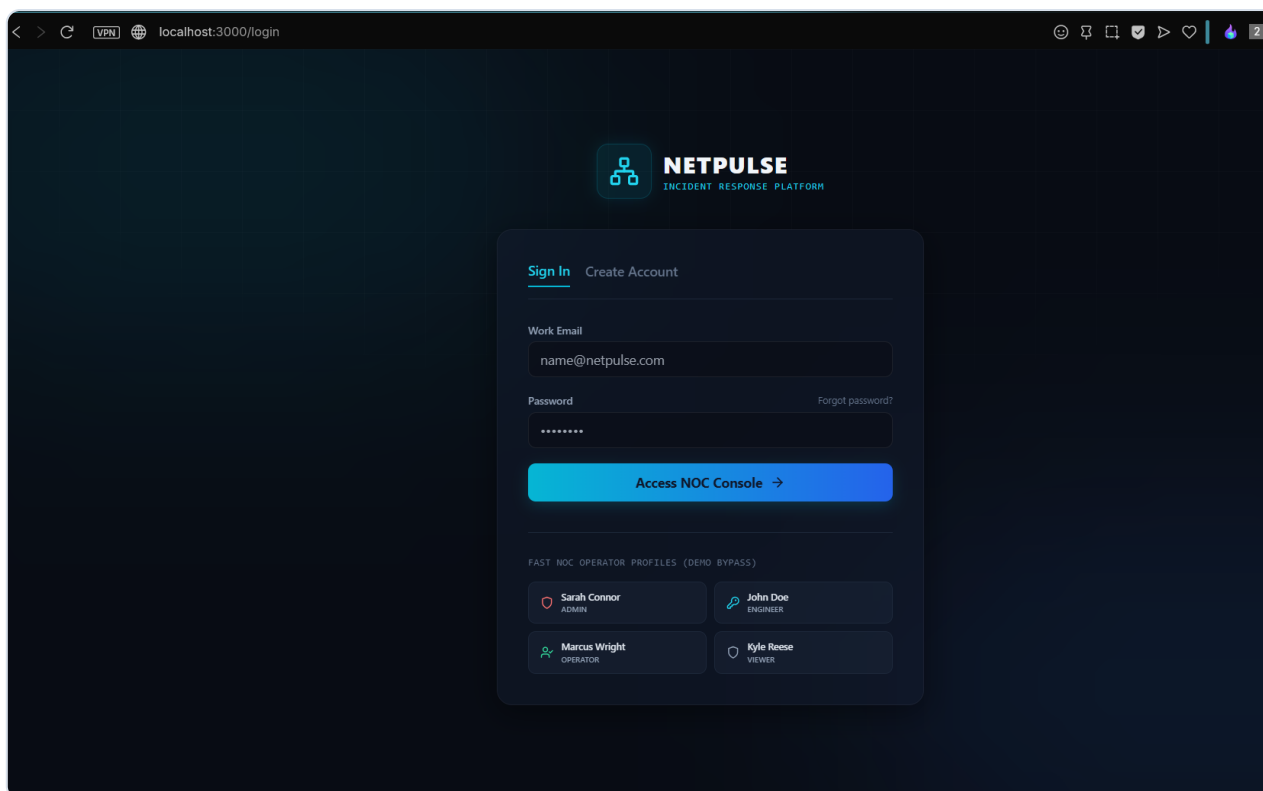
[nodemon] 3.1.14
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): src\**\*
[nodemon] watching extensions: ts
[nodemon] starting 'ts-node src/index.ts'
Connecting to Redis at redis://localhost:6379...
Redis connection failed (AggregateError). NetPulse running with in-memory caching/PubSub.
[Monitoring Engine] Starting NetPulse monitoring scheduler (10s intervals)...
[SLA Monitor] Starting SLA compliance scanning (30s intervals)...
=====
NetPulse Enterprise Backend Server Run
Port: 5000
Environment: Production-Ready
=====
[RCA Engine] Device hq-dist-switch-01 raised CRITICAL alert. Root cause: hq-dist-switch-01 (Correlated: false)
[Alert Correlation] Linking alert ID 9d16f6ad-19c5-4589-a832-546af412e52a on hq-dist-switch-01 to existing incident INC-0002
```

Backend console — monitoring engine startup, Redis fallback, and live RCA/alert-correlation logging

◆ 5.2 — Login Page and Demo Bypass Profiles

The entry point is a dark-themed sign-in screen implemented in `Login.tsx`, backed by `/api/auth` on the backend and issuing JWT access and refresh tokens on success. Beneath the standard email/password form, the login page exposes four Fast NOC Operator Profiles (demo bypass) buttons that let a reviewer sign straight in as one of the four seeded roles — Sarah Connor (ADMIN), John Doe (ENGINEER), Marcus Wright (NOC_OPERATOR), and Kyle Reese (VIEWER) — without typing credentials, useful for quickly demonstrating how the UI's available actions change per role.

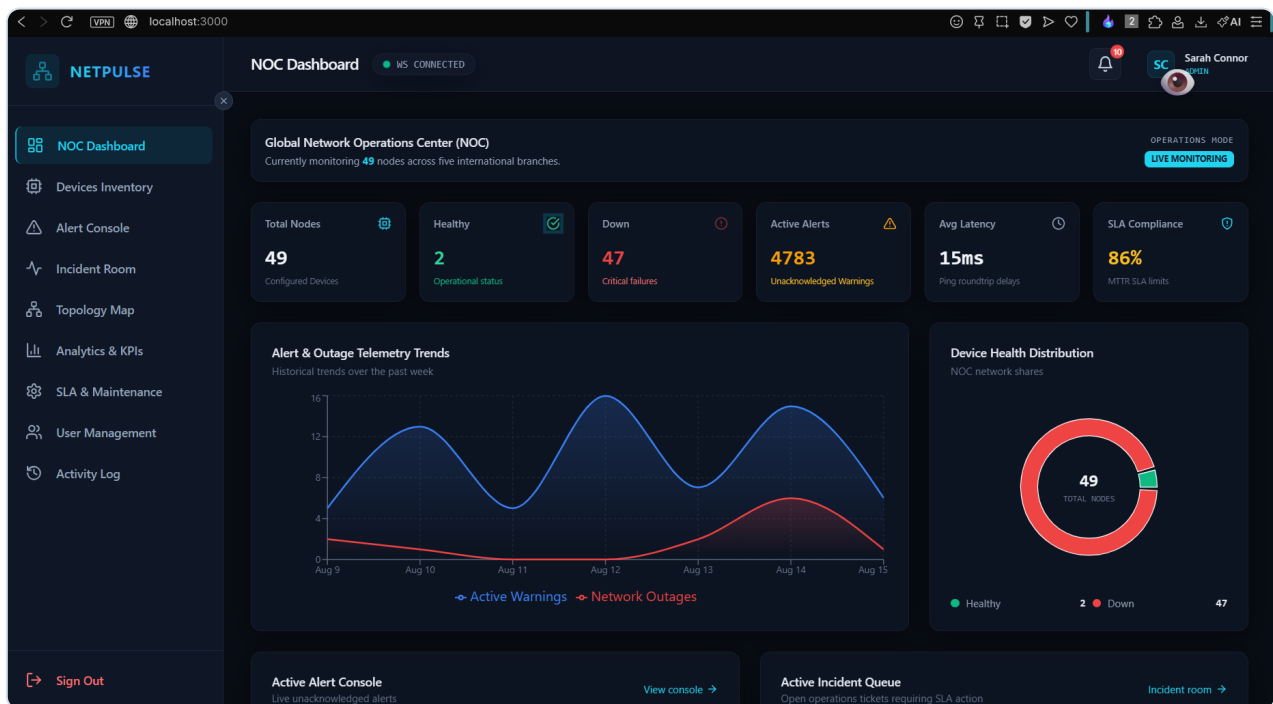
Behind the scenes, each of these seeded accounts uses the same password (`password123`) in the development seed data, and `AuthContext.tsx` persists the resulting session so that role-gated UI elements render correctly for the signed-in profile.



Login page with work-email sign-in and four demo-bypass operator profiles

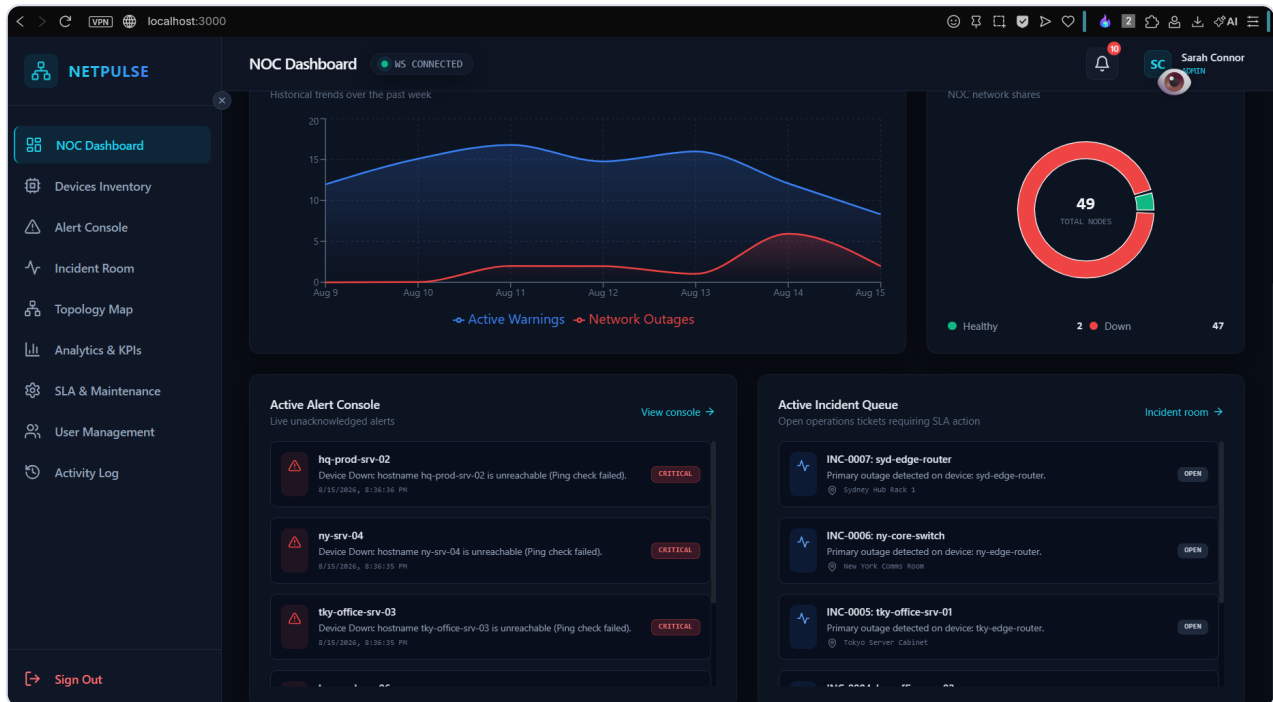
◆ 5.3 — NOC Dashboard Overview

The NOC Dashboard (`Dashboard.tsx`) is the first screen an operator sees after signing in. It states the current operations mode (Live Monitoring) and the total node count under management, then surfaces six headline tiles — Total Nodes, Healthy, Down, Active Alerts, Avg Latency, and SLA Compliance. In this capture the platform is tracking 49 total nodes across five branches, with 2 healthy and 47 down, 4,783 unacknowledged active alerts, 15ms average latency, and 86% SLA compliance against the configured MTTR target.



NOC Dashboard — headline KPI tiles, telemetry trend chart, and device health distribution

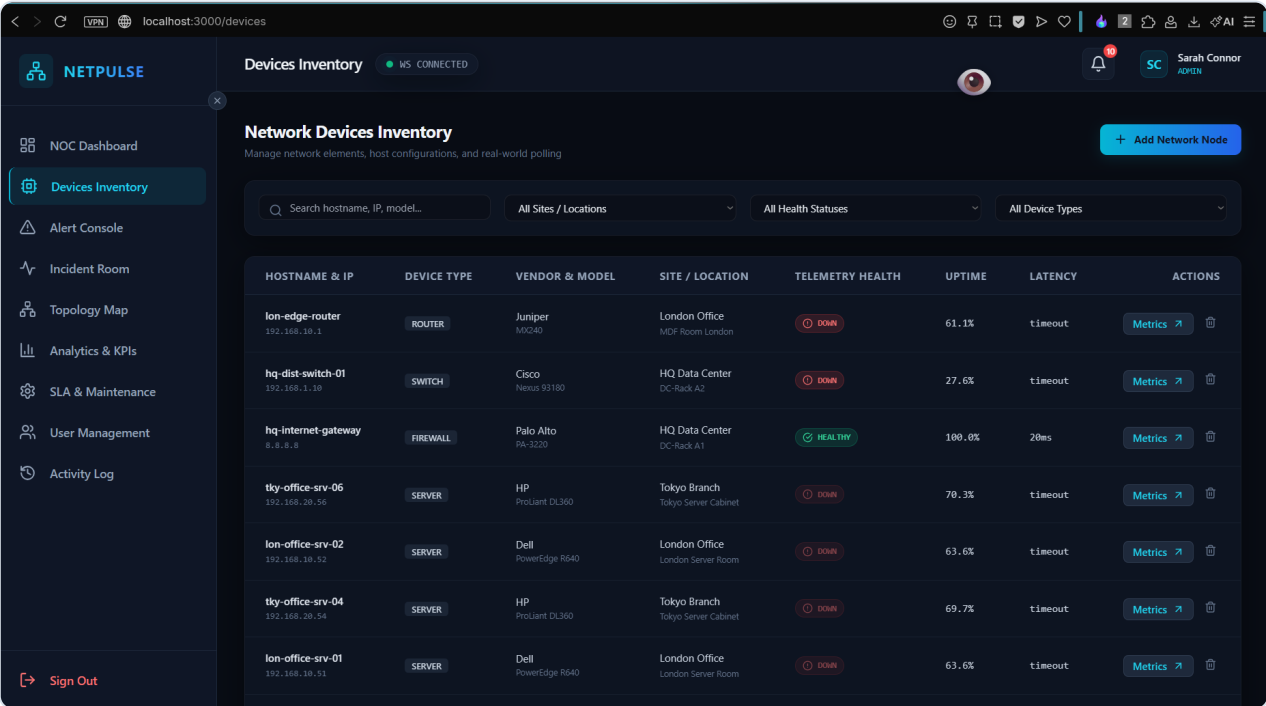
Below the tiles, an Alert & Outage Telemetry Trends area chart plots active warnings against confirmed network outages over the trailing week, and a Device Health Distribution donut chart shows the healthy/down split at a glance. The lower half of the dashboard surfaces two live, auto-scrolling feeds — the Active Alert Console and the Active Incident Queue — both updating immediately over WebSocket as new events arrive.



NOC Dashboard — live Active Alert Console and Active Incident Queue feeds

◆ 5.4 — Devices Inventory

The Devices Inventory (Devices.tsx) lists every device registered in the platform with hostname and IP, device type, vendor and model, site/location, a live telemetry-health badge, rolling uptime percentage, and current latency, backed by GET /api/devices. Free-text search across hostname/IP/model is combined with dropdown filters for site, health status, and device type.



The screenshot displays the NetPulse web application interface. The left sidebar contains navigation links: NOC Dashboard, Devices Inventory (active), Alert Console, Incident Room, Topology Map, Analytics & KPIs, SLA & Maintenance, User Management, and Activity Log. The main content area is titled "Devices Inventory" and includes a sub-header "Network Devices Inventory" with the description "Manage network elements, host configurations, and real-world polling". A search bar and filters for "All Sites / Locations", "All Health Statuses", and "All Device Types" are present. Below these is a table listing network devices.

HOSTNAME & IP	DEVICE TYPE	VENDOR & MODEL	SITE / LOCATION	TELEMETRY HEALTH	UPTIME	LATENCY	ACTIONS
lon-edge-router 192.168.10.1	ROUTER	Juniper MQ40	London Office MDF Room London	DOWN	61.1%	timeout	Metrics ↗
hq-dist-switch-01 192.168.1.10	SWITCH	Cisco Nexus 93180	HQ Data Center DC-Rack A2	DOWN	27.6%	timeout	Metrics ↗
hq-internet-gateway 8.8.8.8	FIREWALL	Palo Alto PA-3220	HQ Data Center DC-Rack A1	HEALTHY	100.0%	20ms	Metrics ↗
tky-office-srv-06 192.168.20.56	SERVER	HP ProLiant DL360	Tokyo Branch Tokyo Server Cabinet	DOWN	70.3%	timeout	Metrics ↗
lon-office-srv-02 192.168.10.52	SERVER	Dell PowerEdge R640	London Office London Server Room	DOWN	63.6%	timeout	Metrics ↗
tky-office-srv-04 192.168.20.54	SERVER	HP ProLiant DL360	Tokyo Branch Tokyo Server Cabinet	DOWN	69.7%	timeout	Metrics ↗
lon-office-srv-01 192.168.10.51	SERVER	Dell PowerEdge R640	London Office London Server Room	DOWN	63.6%	timeout	Metrics ↗

Devices Inventory — searchable, filterable catalogue of monitored network devices

Each row exposes a Metrics action that opens the device's historical CPU, memory, disk, latency, and packet-loss charts (DeviceDetails.tsx), and an Admin-only delete action for decommissioning a node. The DOWN badges visible in this capture correspond directly to the CRITICAL device-down alerts feeding the Alert Console and Incident Queue elsewhere in the platform — inventory, alerting, and incident layers all read from the same Device and Alert tables, so device state stays consistent across screens.

◆ 5.5 — Adding New Network Nodes

New devices are registered through the Add Network Node form (POST /api/devices), which captures the core identity fields — hostname, IP address or DNS target, device type, site placement, vendor, model, and rack/location detail — alongside an Advanced Polling & Agent Parameters section. This is where the monitoring engine's per-device configuration gets set: a Parent Node selector wires the device into the dependency tree the RCA engine walks (`findHighestFailedAncestor`), an optional NetPulse Agent Token switches the device from network-probe monitoring to agent-reported telemetry, and individual toggles enable ICMP ping latency monitoring, comma-separated TCP port checks, an HTTP health-check URL, and a DNS lookup target.

A final Assigned Support Engineer field lets the device be pre-claimed at registration time. Once saved, the device is picked up by the monitoring engine on its very next ten-second poll cycle.

Register Network Element & Configure Polling

Hostname: e.g. hq-core-router-02 | IP Address / DNS Target: e.g. 192.168.1.5

Device Type: ROUTER | Site Placement: Select Site Placement

Vendor: e.g. Cisco | Model: e.g. Catalyst 9300 | Location Details: e.g. Rack A1

ADVANCED POLLING & AGENT PARAMETERS

Parent Node (Dependency RCA Tree): No parent (Internet Gateway / Core Root) | NetPulse Agent Token (Optional): e.g. agent-token-hq-srv-02

☒ Enable ICMP Ping latency monitoring

TCP Port Checks (comma-separated): e.g. 22,80,443

HTTP Url Health Check Target: e.g. http://google.com/health

DNS Lookup Check Target Hostname: e.g. github.com

Assigned Support Engineer: Claim / Unassigned

Buttons: Cancel, Register Node

Add Network Node — device identity fields and advanced polling / RCA-parent configuration

◆ 5.6 — Alert Console — Live Alarm Monitoring

The Alert Console (Alerts.tsx, backed by /api/alerts) is the detailed, action-oriented view of every alarm the monitoring engine has raised, independent of whether it has been folded into an incident. Each row shows the timestamp, severity (Critical/High/Medium/Low), the originating device hostname and IP, a human-readable description generated by the monitoring engine, an escalation level, current status, and operator assignment, with inline Acknowledge, Resolve, and Escalate actions plus a maintenance-suppress shortcut.

The header's connection pill — WS CONNECTED or WS OFFLINE — reflects the live state of the Socket.IO connection maintained by SocketContext.tsx; when connected, newly correlated alerts appear at the top of the table without a manual refresh.

The screenshot displays the NetPulse Alert Console interface. The left sidebar contains navigation links: NOC Dashboard, Devices Inventory, Alert Console (active), Incident Room, Topology Map, Analytics & KPIs, SLA & Maintenance, User Management, and Activity Log. The main area is titled 'Live NOC Alarm Console' and shows a table of active alarms. The table has columns for Timestamp, Severity, Device Hostname, Description, Escalation, Status, Operator Assignment, and Operational Actions. Four critical alarms are listed, all with a status of 'ACTIVE' and 'Unassigned' operator. Each row includes 'Acknowledge' and 'Resolve' buttons, along with an escalation arrow and a suppression icon.

TIMESTAMP	SEVERITY	DEVICE HOSTNAME	DESCRIPTION	ESCALATION	STATUS	OPERATOR ASSIGNMENT	OPERATIONAL ACTION
8/15/2026, 9:25:11 PM	CRITICAL	hq-prod-srv-08 192.168.2.18	Device Down: hostname hq-prod-srv-08 is unreachable (Ping check failed).	Lv. 1	ACTIVE	Unassigned	Acknowledge Resolve ↑
8/15/2026, 9:25:11 PM	CRITICAL	hq-prod-srv-02 192.168.2.12	Device Down: hostname hq-prod-srv-02 is unreachable (Ping check failed).	Lv. 1	ACTIVE	Unassigned	Acknowledge Resolve ↑
8/15/2026, 9:25:10 PM	CRITICAL	hq-dist-switch-02 192.168.1.11	Device Down: hostname hq-dist-switch-02 is unreachable (Ping check failed).	Lv. 1	ACTIVE	Unassigned	Acknowledge Resolve ↑
8/15/2026, 9:25:09 PM	CRITICAL	hq-prod-srv-07 192.168.2.17	Device Down: hostname hq-prod-srv-07 is unreachable.	Lv. 1	ACTIVE	Unassigned	Acknowledge Resolve ↑

Alert Console — live alarm feed with severity, escalation level, and inline operator actions

◆ 5.7 — Incident Room — RCA Diagnosis and Workflow

Opening an incident from the queue lands on the Incident Room (IncidentDetails.tsx, backed by /api/incidents/:id), where the platform's RCA output becomes actionable. The RCA Diagnosis panel names the identified root-cause device, the impacted services, and the raw root-cause detail string generated at correlation time; the Affected Devices List enumerates every device the alert-correlation engine has linked to this incident.

The screenshot displays the NetPulse Enterprise Incident Room interface for Incident INC-0007 Room. The interface is divided into several sections:

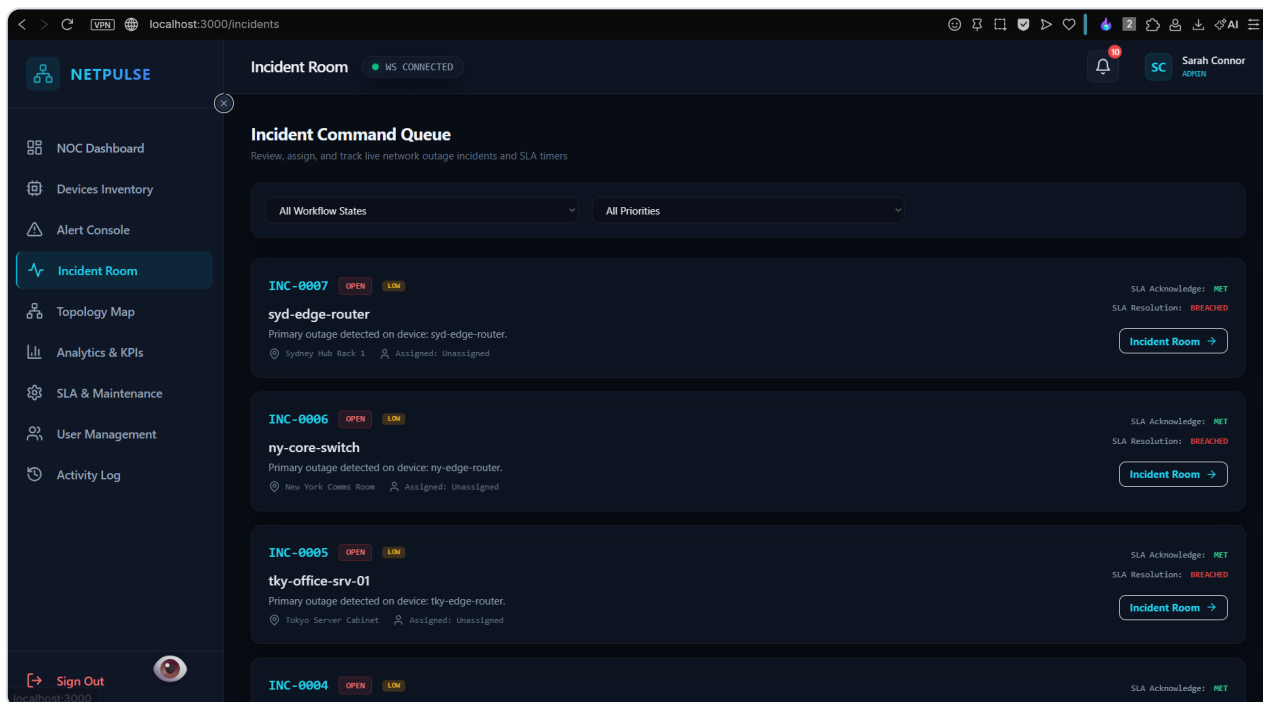
- Left Sidebar:** Contains navigation links for NOC Dashboard, Devices Inventory, Alert Console, Incident Room, Topology Map, Analytics & KPIs, SLA & Maintenance, User Management, and Activity Log. A 'Sign Out' button is at the bottom.
- Header:** Shows 'NetPulse Enterprise' with a 'WS CONNECTED' status and a user profile for Sarah Connor ADEN.
- Incident Details:** Displays 'Incident INC-0007 Room' with a priority of 'LOW' and an 'SLA BREACHED' status. It includes an 'OPEN' button.
- RCA DIAGNOSIS:** Identifies the root cause device as 'syd-edge-router' and lists impacted services: 'All Site-to-Site WAN traffic and external internet access.' It also shows a primary outage detected on the device.
- NOC WORKFLOWS:** Features buttons for 'Set RESOLVED' and 'Close Ticket', along with an 'Advance Status Workflow' dropdown set to 'OPEN'.
- AFFECTED DEVICES LIST:** Lists affected devices, including 'syd-edge-router' and 'syd-core-switch'.
- ENGINEERING DISCUSSION:** A section for logging diagnostic findings, currently showing 'No entries. Post comments to log diagnostic findings.' with a text input field and a 'Post' button.
- AUDITABLE OPERATIONS TIMELINE:** A chronological list of events, including correlated events for device failures and outages on 'syd-edge-router', 'syd-srv-02', and 'syd-srv-05'.

Incident Room — RCA diagnosis, affected-device list, workflow controls, and operations timeline

The NOC Workflows panel exposes the incident's status transitions — an Advance Status Workflow dropdown covering the full OPEN → INVESTIGATING → IDENTIFIED → FIX_IN_PROGRESS → MONITORING → RESOLVED lifecycle. An Engineering Discussion thread lets responders post diagnostics, log excerpts, and resolution notes that are timestamped and attributed. Every state change, correlated alert, and SLA event is also written to an Auditable Operations Timeline, giving a full, tamper-evident record of how the incident was handled from first detection to closure.

◆ 5.8 — Incident Command Queue

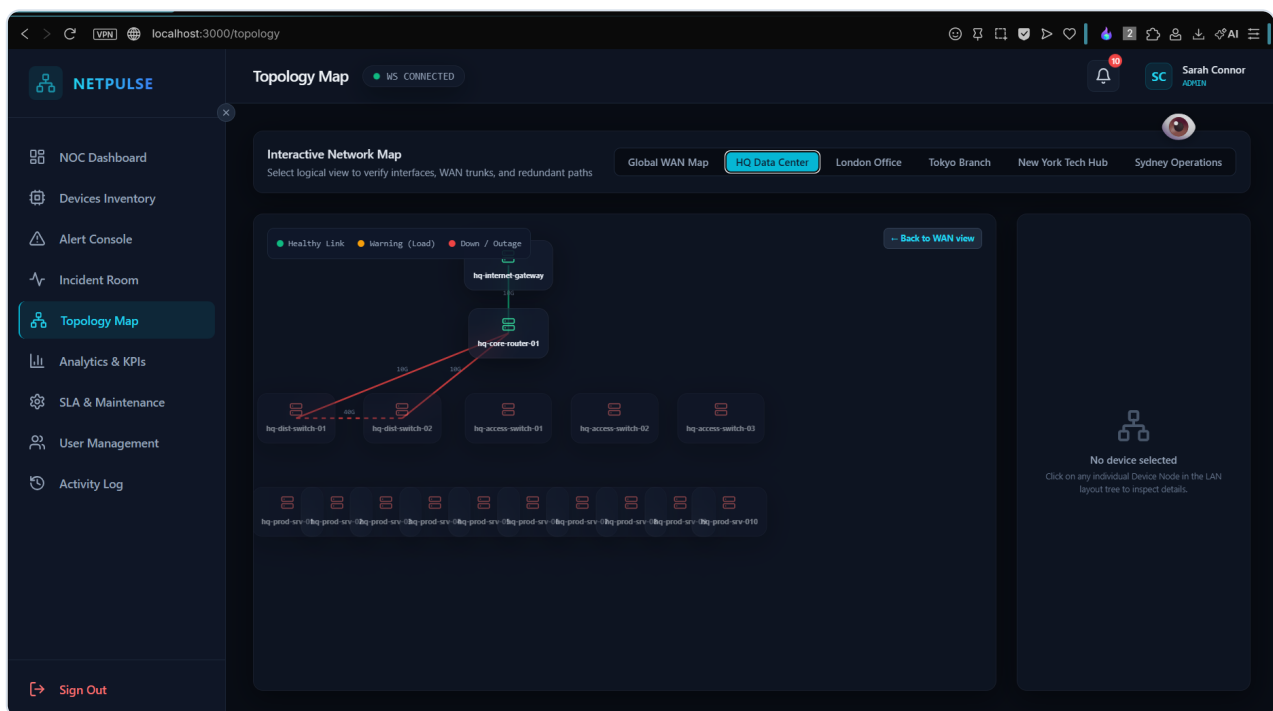
The Incident Command Queue (Incidents.tsx) is the triage-level view across all incidents. Workflow-state and priority filters sit above a card list where each incident shows its identifier (INC-0001, INC-0002, ...), current status, priority, the root-cause device and a short description, site/location, assignment, and — critically — separate SLA Acknowledge and SLA Resolution indicators showing Met or Breached against the incident's assigned SlaPolicy.



Incident Command Queue — all open incidents with live SLA acknowledge/resolution status

◆ 5.9 — Topology Map

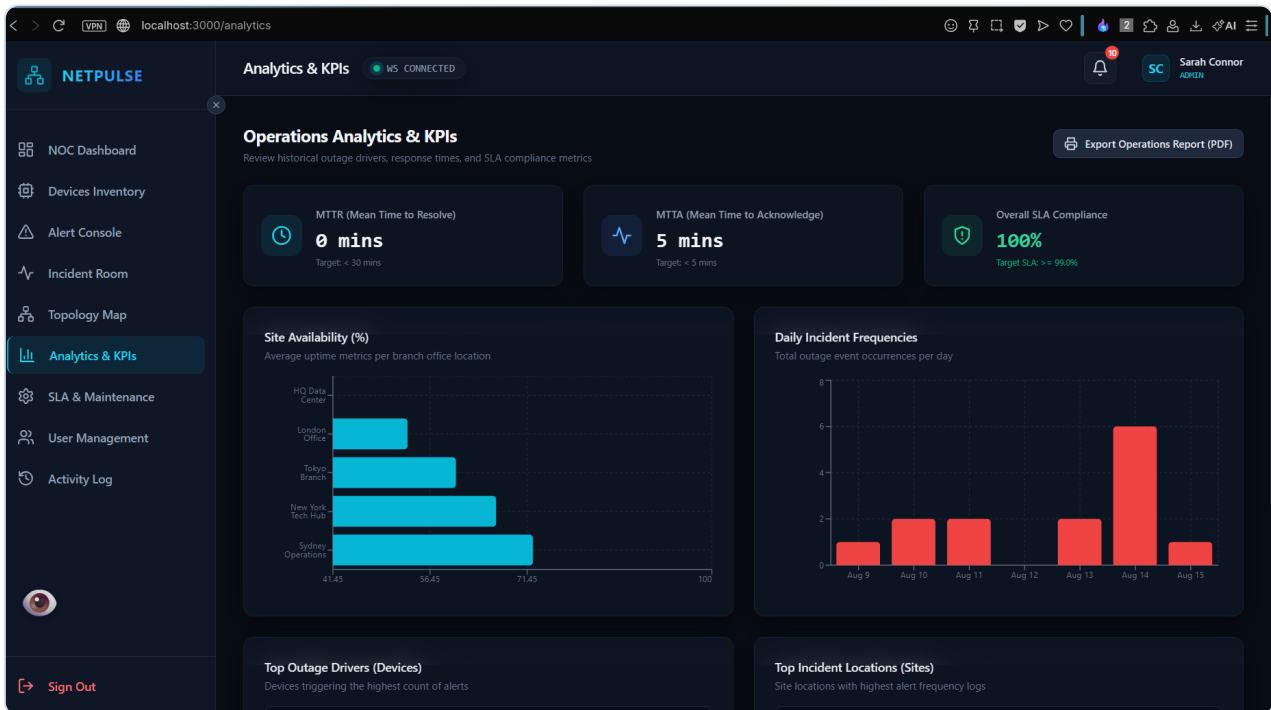
The Topology Map (Topology.tsx, backed by /api/topology) renders the network as an interactive diagram rather than a flat table. A Global WAN Map view shows site-to-site connectivity across all five branches, and per-site tabs drill down into that site's LAN tree, from internet gateway and core router down through distribution and access switches to the production servers behind them. Link color encodes health in real time: green for a healthy link, amber for a link under load/warning, and red for a link that is down or carrying an active outage.



Topology Map — HQ Data Center LAN view showing a red (down) link path from the core router

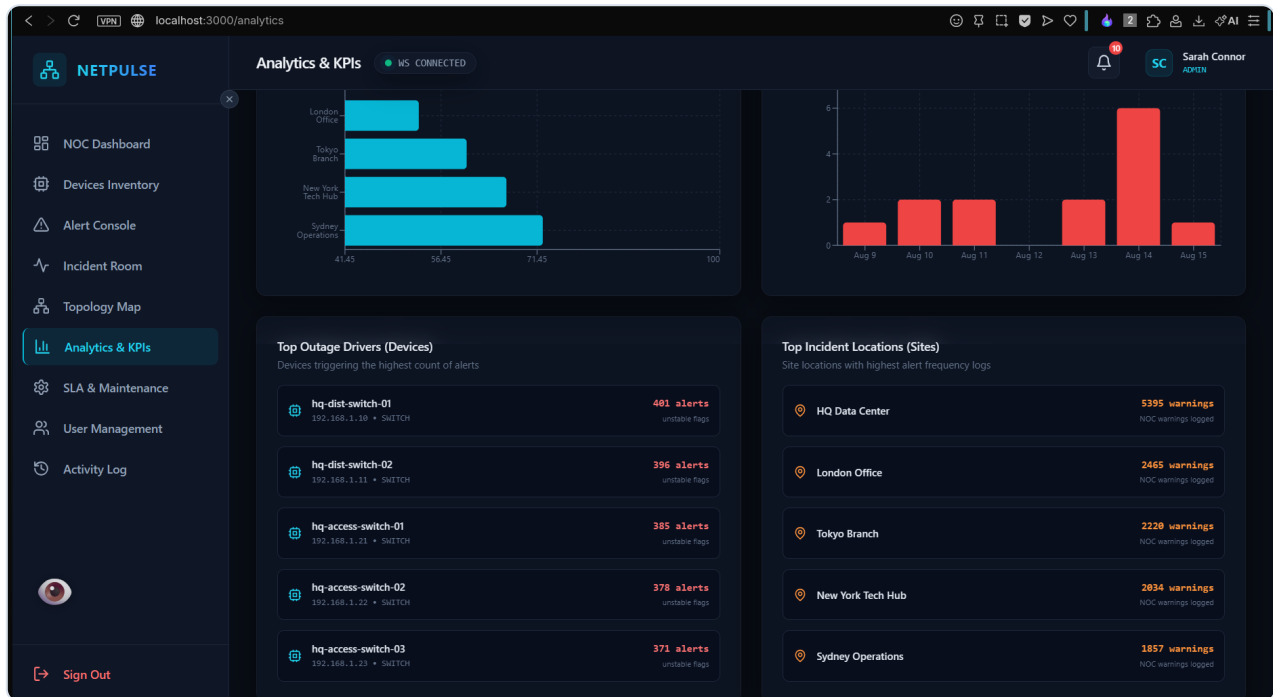
◆ 5.10 — Analytics & KPIs

The Analytics & KPIs page rolls incident and alert history up into the metrics a NOC actually reports on: Mean Time to Resolve (MTTR), Mean Time to Acknowledge (MTTA), and overall SLA compliance percentage, each shown against its configured target. An Export Operations Report (PDF) action lets these figures be pulled out for a shift handover or stakeholder update.



Analytics & KPIs — MTTR, MTTA, SLA compliance, and per-site availability

Below the headline KPIs, a Site Availability chart ranks the five branches by average uptime, and a Daily Incident Frequencies chart plots outage counts per day over the trailing week. Two further panels identify Top Outage Drivers (Devices) and Top Incident Locations (Sites) — exactly the kind of hot-spot analysis a real NOC uses to prioritize hardware replacement or network redesign work.



Analytics & KPIs — top outage-driving devices and top incident locations by site

◆ 5.11 — SLA & Maintenance

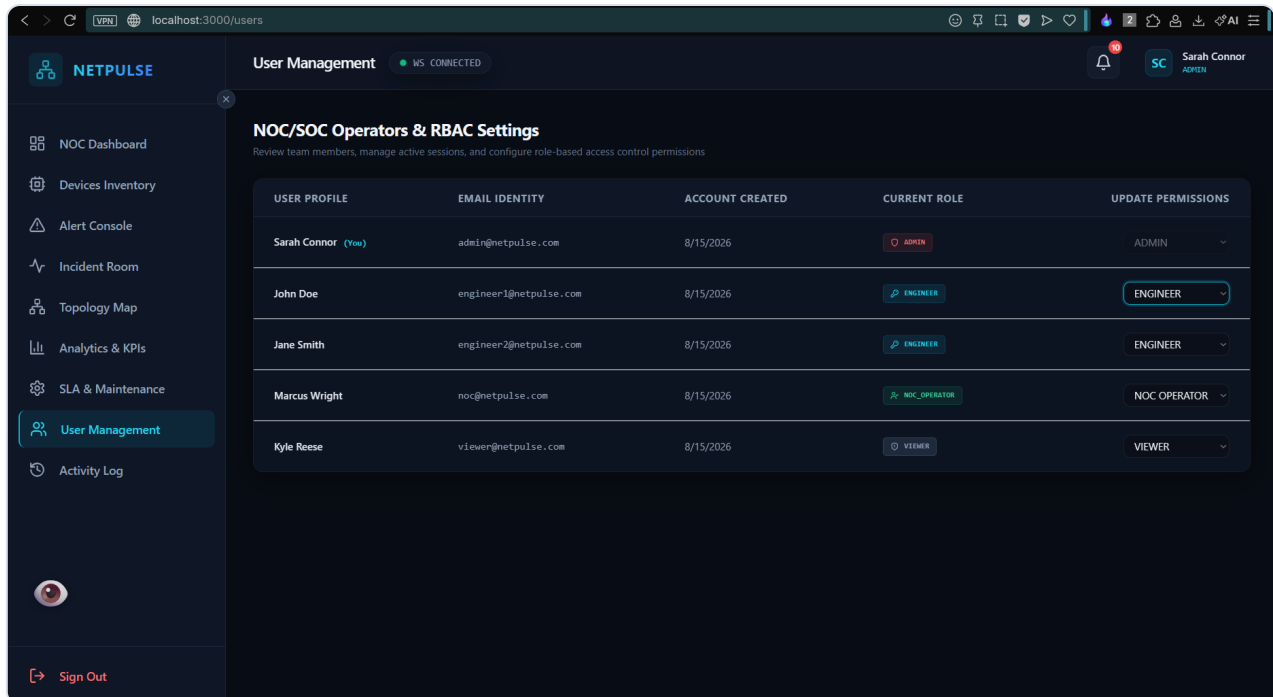
The SLA & Maintenance page is where the thresholds enforced by the backend's thirty-second SLA scanner are configured. Four Active SLA Policies — Critical, High, Medium, and Low — each define an acknowledgement time limit and a resolution time limit. A Schedule Maintenance panel lets an operator pick a target device, name the maintenance reason, and set a start/end window; while active, the monitoring engine flags the device as MAINTENANCE and suppresses new alerts and incident creation for it.

The screenshot shows the NetPulse web interface at localhost:3000/settings. The left sidebar contains navigation links: NOC Dashboard, Devices Inventory, Alert Console, Incident Room, Topology Map, Analytics & KPIs, SLA & Maintenance (selected), User Management, and Activity Log. The main content area is titled 'SLA & Maintenance' with a 'WS CONNECTED' status. Below this is the 'SLA Policies & Maintenance Windows' section, which includes a subtitle 'Configure service response limits and schedule offline windows for hardware upgrades'. The 'Active SLA Policies' section lists four policies: Critical Priority Policy (Ack Time Limit: 5 mins (300s), Res Time Limit: 30 mins (1800s), CRITICAL), High Priority Policy (Ack Time Limit: 15 mins (900s), Res Time Limit: 120 mins (7200s), HIGH), Low Priority Policy (Ack Time Limit: 60 mins (3600s), Res Time Limit: 1440 mins (86400s), LOW), and Medium Priority Policy (Ack Time Limit: 30 mins (1800s), Res Time Limit: 480 mins (28800s), MEDIUM). Each policy has a 'Modify Policy Parameters' link. The 'Schedule Maintenance' section on the right allows setting time bounds to suppress telemetry alarms during updates. It includes a 'Target Network Element' dropdown (labeled 'Select device...'), a 'Schedule Name / Reason' text input (e.g., 'Firmware patch upgrade v2.4'), and two date-time pickers for 'Maintenance Window Start' and 'Maintenance Window End' (both in 'dd-mm-yyyy --:--' format).

SLA & Maintenance — priority-based SLA policies and maintenance-window scheduling

◆ 5.12 — User Management and RBAC

User Management (Users.tsx, backed by /api/users) lists every account with its user profile, email identity, account-creation date, and current role, and lets an Admin change any user's role through an inline dropdown. NetPulse implements four roles end-to-end — ADMIN, ENGINEER, NOC_OPERATOR, and VIEWER — with `authMiddleware.ts` enforcing role checks on sensitive routes server-side, not just in the UI.



User Management — operator roster with role-based access control settings

◆ 5.13 — Activity Log — System Audit Trails

The Activity Log is the system-wide audit trail: every authenticated action — logins, configuration changes, incident status transitions, SLA breach events, and device registrations — is written to the ActivityLog table with a timestamp, module tag, a human-readable audit action, the acting operator's name and email, and their source IP address.

NetPulse Enterprise NOC

Activity Log WS CONNECTED

System Audit Trails & Logs

Review chronological records of configuration changes, user logins, and incident updates

Search action details, operator emails... All System Modules

TIMESTAMP	MODULE	AUDIT ACTION	OPERATIONS DETAILS	OPERATOR NAME	SOURCE IP
8/15/2026, 11:50:46 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 9:49:36 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 9:48:14 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 9:24:52 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 8:55:25 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 8:36:12 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 8:30:07 PM	AUTH	User Login	Logged in from IP ::1	Sarah Connor admin@netpulse.com	::1
8/15/2026, 8:25:46 PM	AUTH	User Login	Logged in from IP ::1	John Doe jdoe@netpulse.com	::1

Sign Out

Activity Log — chronological audit trail of authentication and configuration events

Challenges & Resolutions

Four engineering problems that came up while building NetPulse, and how each one was solved.

Challenges Faced and How I Resolved Them

01 WebSocket Connection Management

PROBLEM

Because every alert, incident update, and metric reading is meant to reach the browser over Socket.IO in real time, a dropped or slow-to-reconnect WebSocket connection meant operators could silently miss critical alerts while believing the dashboard was current — the exact failure mode a NOC tool cannot afford. Network hiccups, backend restarts during development, and the browser tab losing focus all triggered disconnects.

RESOLUTION

I centralized all socket lifecycle handling in the frontend's `SocketContext.tsx` so every page shares one managed connection instead of opening its own. The context surfaces the live connection state directly in the UI as the WS CONNECTED / WS OFFLINE pill. On the backend, `socketManager.ts` centralizes every broadcast call so a reconnecting client is guaranteed to be resubscribed to the same event set rather than a partial one.

✓ RESOLVED

02 Alert Correlation and Deduplication

PROBLEM

In an early version of the monitoring engine, every device that failed its check simply opened its own incident. When a distribution switch or core router went down, every device behind it also failed its checks on the very next ten-second poll, which meant a single physical outage could open dozens of independent incidents and flood the Alert Console.

RESOLUTION

I built a dependency-aware correlation and RCA engine in `alertCorrelation.ts`. Every device can declare a `parentDeviceId`, forming a tree that mirrors the real network topology. When a new alert comes in, `findHighestFailedAncestor()` walks that device's parent chain looking for the highest ancestor already DOWN; if one exists, the new alert is attached to that ancestor's existing open incident instead of spawning a new one. Only genuinely independent failures open a brand-new incident via `generateNextIncidentId()`.

✓ RESOLVED

03 SLA Monitoring and Breach Detection

PROBLEM

Tracking SLA compliance accurately requires comparing an incident's real elapsed time against a policy threshold continuously, not just at the moment a user happens to load the page — otherwise a breach that occurs while no one is looking at the Incident Room would never surface until someone manually checked.

RESOLUTION

The SLA monitor is implemented as its own background scheduler (`startSlaPolicyChecker`) rather than a page-load calculation. Every thirty seconds it loads every incident still active, computes elapsed age against policy thresholds, and the first time either is crossed it flags the breach, writes a descriptive IncidentTimeline entry, and rebroadcasts the incident over WebSocket so it appears live without anyone needing to refresh.

✓ RESOLVED

04 Local Development Without Docker's Postgres and Redis

PROBLEM

The production topology assumes PostgreSQL and Redis running as Docker containers, but developing and demoing the platform on a machine without Docker running meant the backend needed to survive both a missing database engine and a missing cache/pub-sub broker without simply crashing on boot.

RESOLUTION

Prisma's datasource makes the SQLite-vs-PostgreSQL choice a configuration concern rather than a code concern, so the same schema and query code run unmodified against a local SQLite file. For the cache/pub-sub layer, redisClient.ts wraps every Redis operation in a connection-state check and falls back to an in-memory Map plus a Node EventEmitter — a trade-off that's correct for a single-instance dev/demo run but would need Redis restored for horizontally scaled production.

✓ RESOLVED

Conclusion & Key Takeaways

What NetPulse demonstrates end-to-end, and the single biggest lesson from building it.

Conclusion and Key Takeaways

NetPulse Incident Response Platform demonstrates a complete, working implementation of the workflow that sits at the heart of every enterprise Network Operations Center: Detect → Classify → Assign → Resolve. Devices are polled continuously and honestly (real ICMP, TCP, HTTP, and DNS checks, or live agent telemetry), failures are automatically correlated back to a single root cause using a dependency-tree RCA engine rather than left to flood the queue individually, incidents are tracked against configurable SLA policies with automatic breach detection, and every action taken along the way is written to a permanent audit trail.

In its current dataset the platform monitors 49 nodes across five simulated branches — HQ Data Center, London Office, Tokyo Branch, New York Tech Hub, and Sydney Operations — exercising every layer of the stack: React/TypeScript on the frontend, Node.js/Express/TypeScript on the backend, Prisma over PostgreSQL (with SQLite fallback), Redis for caching and pub/sub (with an in-memory fallback), and Socket.IO for real-time delivery.

Technology Stack Summary

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, TanStack React Query, Recharts, Framer Motion, Lucide Icons
Backend	Node.js, Express, TypeScript, Prisma ORM, JWT authentication, Helmet, express-rate-limit
Real-time layer	Socket.IO (WebSocket) server and client
Database	PostgreSQL (Docker) with automatic SQLite fallback for local development
Cache / PubSub	Redis with automatic in-memory (Map + EventEmitter) fallback
Agent	Standalone NetPulse Agent daemon (netpulse-agent.ts) for host-level metrics
Deployment	Docker Compose (Postgres, Redis, Express backend, Nginx-hosted React frontend)

NetPulse is, by design, a simplified, self-built analogue of the category occupied by tools like ServiceNow ITOM, PagerDuty, Datadog, and SolarWinds — it does not attempt to match their scale, integration ecosystems, or hardened multi-tenant security posture, and its network data is simulated rather than sourced from production hardware. What it does demonstrate is that the

underlying architectural problems those products solve — dependable real-time delivery, correct alert correlation instead of alert flooding, policy-driven SLA enforcement, and a fully auditable operations record — can be understood deeply enough to implement from first principles in a single, coherent codebase.

The single biggest personal takeaway from building NetPulse end-to-end was how much of a production incident-management platform's value lives in the correlation and workflow layers rather than in monitoring itself: raw uptime checks are the easy part; deciding what to do with a hundred simultaneous failures, and giving a human operator one clear signal instead of a hundred noisy ones, is where the real engineering is.

Future Plans

Five directions planned for NetPulse beyond its current demo/test configuration.



PHASE 01 · HARDWARE MONITORING

SNMP Integration

Monitor real physical network hardware rather than simulated endpoints.

PLANNED



PHASE 02 · INTELLIGENCE

ML-Based Anomaly Detection

Flag abnormal metric patterns before they cross a hard threshold.

PLANNED



PHASE 03 · MOBILITY

Companion Mobile App

Let on-call engineers acknowledge and work incidents away from a desktop.

PLANNED



PHASE 04 · ESCALATION

Slack & PagerDuty Integrations

Wire up escalation and on-call paging to the tools NOC teams already use.

PLANNED



PHASE 05 · SCALE

Hardened AWS Cloud Deployment

A horizontally scaled deployment with managed Postgres, managed Redis, and a multi-instance backend behind a load balancer.

PLANNED

Glossary & Acronyms

Terms and acronyms used throughout this report.

NOC	Network Operations Center — the team/console that monitors and manages network health.	SOC	Security Operations Center — the security-focused counterpart to a NOC.
RCA	Root-Cause Analysis — identifying the originating failure behind a set of symptoms.	SLA	Service Level Agreement — a policy defining acceptable response/resolution times.
MTTR	Mean Time to Resolve — average time from incident open to resolution.	MTTA	Mean Time to Acknowledge — average time from alert raised to acknowledgement.
RBAC	Role-Based Access Control — permissions granted according to a user's assigned role.	WAN	Wide Area Network — connectivity spanning multiple sites or geographies.
LAN	Local Area Network — the network within a single site or building.	JWT	JSON Web Token — a signed token format used for authentication sessions.
ORM	Object-Relational Mapper — maps application objects to database rows (Prisma).	API	Application Programming Interface — the contract backend services expose to clients.
REST	Representational State Transfer — the HTTP API style used by NetPulse's backend.	DNS	Domain Name System — resolves hostnames to IP addresses; used in device health checks.
ICMP	Internet Control Message Protocol — the basis of the ping check used for reachability.	TCP	Transmission Control Protocol — used for port-level reachability checks.
ITOM	IT Operations Management — the software category NetPulse's architecture is modeled on.	ITSM	IT Service Management — the discipline of managing IT as a set of services, including incident management.

ABOUT THIS REPORT

NetPulse Incident Response Platform is a personal, self-built project created to demonstrate full-stack and infrastructure engineering ability. Network data shown throughout this report is simulated for demonstration purposes.